

OADM: Optimally Approximate Floating-Point Divider-Multiplier with Runtime Configurability

Abstract—Approximate computing is a promising alternative to improve energy efficiency for IoT devices on the edge. By use of tangent planes, this work mainly proposes an optimally approximated floating-point divider with runtime configurability. We turn the division approximation to an optimal problem and obtain a theoretically sound formula, which is similar to the formula of the approximated floating-point multiplier in [1]. With the approximate formula, a multi-level architecture is proposed to implement the run-time configurability and the proposed divider strikes an advantageous balance between accuracy and resource efficiency comparing to the prior state-of-the-art approximate dividers. Furthermore inspired by the similarities observed in [1], we propose a unified hardware architecture supporting both approximate floating-point division and multiplication. The proposed unified divide-multiply unit can be configured to execute either approximate division or approximate multiplication in circuit with simple logic operations.

Index Terms—tangent plane, approximate computation, divider, multiplier

I. INTRODUCTION

Energy efficiency has emerged as one of most critical criteria in computing systems design in recent years. Though in general computing systems are designed to generate accurate outputs, applications in domains of computer vision and machine learning can tolerate amounts of inaccuracies in their computations. Computational accuracy is traded for reductions in power consumption and design complexity [2] [3].

In approximate hardware design, hardware elements are simplified to reduce the power consumption and design area, therefore adding inaccuracies. The implementation of these approximations are mostly focused on basic arithmetic building blocks (e.g. adders and multiplier), due to their high utilization in different applications. These approximate designs can then replace or augment their accurate counterparts in general purpose processors.

While a number of designs for approximate adders and multipliers have been proposed, the approximate design of dividers is not as well investigated. Arithmetic divider is a basic yet complex module in integrated circuits. It plays an increasingly important role in a variety of fields and is widely used in graphic processing. However, divider is often a performance limiting module in these applications due to its complexity. The lack of fast division module in integrated circuits has for a long time been the bottleneck for some real-time multimedia applications. This work aims to address this issue, by proposing OADM (an **O**ptimally **A**pproximated **F**loating-**P**oint **D**ivider-**M**ultiplier). Our main contributions are as follows:

- By use of tangent planes, an optimization formulation is given to optimize the error of the approximate divider

which acts as the basis of the divider architecture; furthermore, we modify the proposed formula and obtain a new formulation with unbiased error distribution.

- Based on the optimization formulation mentioned above, we present a novel multi-level approximate FP divider with run-time configurability. To enhance the accuracy, we can add up the error compensations of increasing levels from the level 0. The circuit for the error compensation of each level is easily implemented with shift, inversion and subtraction. This reduce the cost complexity from exponential to linear function of levels n directly.
- For different levels, we derive mathematical formulation for the absolute error of approximations in the proposed divider by the Taylor's Theorem for two variables.

The experimental results clearly demonstrate that, in comparison to state-of-the-art dividers, OADM strikes an advantageous balance between accuracy and resource efficiency. **(In this paragraph, we should list the experimental results simply. Read [16] for reference)**

The rest of the paper is organized as follows. In Section II, we review some previous related works on the approximated arithmetics; in Section III, we propose the design of approximated divider architecture and give the analytical error analysis in Section IV; in Section V and Section VI, we evaluate the performance of the proposed approximate divider with experimental results and in real applications; finally, we summarize our conclusion in Section VII.

II. PREVIOUS WORK

In this section, we review some of the related prior work done in the area of approximate design. Approximate architecture can enable the designer to introduce inaccuracies in a controlled and deterministic manner.

While a few approximate designs have been proposed for both adders and multipliers [4], [5], [6], [1], dividers and other arithmetic units have not received as much attention. This is due in part to lower utilization of blocks such as dividers compared to adders and multipliers and the main reason of the lower utilization of dividers is in part a result of high cost of dividers in terms of delay and power. As shown in [7], an accurate FP divider incurs more than $15\times$ Area-Delay-Product and over $1.5\times$ power consumption compared to an accurate FP multiplier. Therefore, approximate divider design with significant improvements in these areas can facilitate utilization of these arithmetic blocks.

Recently, several works on approximate dividers have been proposed. These methods have their own advantages, disadvantages and mainly focus on:

(1) truncation-based [8], [9], [10]. The ideas lie on dynamically select the most relevant bits of each of the operands and reduce complexity by discarding the lower, less significant bits. Truncated-based designs can achieve high computational accuracy, but often need a smaller accurate divider design in its core and so impose substantial hardware overhead;

(2) logarithm-based [7], [11], [12]. These designs depend on the approximation $\log_2(1+a) \approx a$ mainly. Since the error of the approximation $\log_2(1+a) \approx a$ is small only if a is close to zero, logarithm-based designs are more hardware-efficient but should balance between the two accuracy and configurability during runtime;

(3) reciprocal-based dividers [13], [14], [15], [16], [17]. These dividers use different ways to approximate the reciprocal $\frac{1}{y}$ of divisor y , such as Taylor's expansions[13], Newton Raphson method[15], quadratic curve fitting[17] and piece-wise linear segments[16];

(4) In [18], the proposed approach is based on the curve fitting method to approximate the curved surfaces with small square planes as follows:

$$\frac{x}{y} \approx p \cdot x - q \cdot y + r.$$

First decide the partition scheme and then use the Matlab curve-fitting toolbox to find the coefficients p, q and r for the square/triangular plane to approximate the individual region of the curved surfaces $\frac{x}{y}$. The Look-up Table stores the coefficients p, q and r . Since there are no concrete information about the constants p, q, r , the proposed architecture relies on pre-loaded lookup tables (LUTs), making runtime adjustments to accuracy impossible.

III. PROPOSED APPROXIMATE FP DIVIDER-MULTIPLIER

Comparing to integer computing, FP arithmetic is more complex and so energy consuming. According to IEEE 754 standard in [20], an FP number F_x consists of sign, mantissa and exponent as follows:

$$F_x = \text{sign}_x \times x \times 2^{E_x}, \forall x \in [1, 2)$$

where sign_x, x, E_x refer to the sign, mantissa and exponent of F_x respectively. Thus for two FP numbers F_x and F_y , the division is:

$$\frac{F_x}{F_y} = \text{sign}_x \text{sign}_y \times \frac{x}{y} \times 2^{E_x - E_y}.$$

In a general processor, for an FP division, the sign bits is computed by an XOR operation and the exponent bits are computed by a subtractor. The division $\frac{x}{y}$ of two mantissas is much more energy consuming and costlier than the computations of signs and exponents. The motivation of this work is to propose an energy efficient divider for $\frac{x}{y}$ between mantissas.

A. First Method

On $[y_1, y_2]$, our aim is to approximate $f(y) = \frac{1}{y}$ by the tangent line $g(y) = -\frac{y}{p^2} + \frac{2}{p}$ of some point p in $[y_1, y_2]$. We try to find a point p which minimizes the error function

$$\begin{aligned} E_I(p) &= \int_{y_1}^{y_2} \left[\frac{1}{y} - \left(-\frac{y}{p^2} + \frac{2}{p} \right) \right] dy \\ &= \ln y_2 - \ln y_1 + \frac{y_2^2 - y_1^2}{2p^2} - \frac{2}{p}(y_2 - y_1). \end{aligned}$$

By

$$0 = E'_I(p) = (y_1^2 - y_2^2)p^{-3} + 2(y_2 - y_1)p^{-2},$$

we obtain $p = \frac{y_1 + y_2}{2}$. Following the symbols in [1], we let $k_1 = \frac{y_1 + y_2}{2}, k_2 = \frac{x_1 + x_2}{2}$. Thus on $[x_1, x_2] \times [y_1, y_2]$, we have

$$\begin{aligned} \frac{x}{y} = x \cdot \frac{1}{y} &\approx x \cdot \left(-\frac{y}{k_1^2} + \frac{2}{k_1} \right) \\ &= -\frac{1}{k_1^2}xy + \frac{2}{k_1}x \\ &\approx -\frac{1}{k_1^2}(-k_1k_2 + k_1x + k_2y) + \frac{2}{k_1}x \\ &= -\frac{1}{k_1^2}(-k_1k_2 + k_1x + k_2y - 2k_1x) \\ &= \frac{1}{k_1^2}(k_1k_2 + k_1x - k_2y), \end{aligned} \quad (1)$$

where the second approximate equation follows the main result in [1]. Set the approximate linear function $z_I = \frac{1}{k_1^2}(k_1k_2 + k_1x - k_2y)$.

B. Second Method

In this subsection, we try to approximate $z = \frac{x}{y}$ on $[x_1, x_2] \times [y_1, y_2]$ by tangent plane directly. For the curved surface $F(x, y, z) = z - \frac{x}{y} = 0$, the tangent plane at the point $(x_0, y_0, z_0 = \frac{x_0}{y_0})$ is:

$$\frac{\partial F}{\partial x} \cdot (x - x_0) + \frac{\partial F}{\partial y} \cdot (y - y_0) + \frac{\partial F}{\partial z} \cdot (z - z_0) = 0,$$

where $\frac{\partial F}{\partial x} = -\frac{1}{y_0}, \frac{\partial F}{\partial y} = \frac{x_0}{y_0^2}, \frac{\partial F}{\partial z} = 1$. So the tangent plane of $z = \frac{x}{y}$ at the point $(x_0, y_0, z_0 = \frac{x_0}{y_0})$ is:

$$-\frac{1}{y_0} \cdot (x - x_0) + \frac{x_0}{y_0^2} \cdot (y - y_0) + 1 \cdot (z - z_0) = 0.$$

That is

$$\begin{aligned} z &= \frac{x_0}{y_0} + \frac{1}{y_0}(x - x_0) - \frac{x_0}{y_0^2}(y - y_0) \\ &= \frac{x_0}{y_0} + \frac{1}{y_0}x - \frac{x_0}{y_0^2}y. \end{aligned}$$

In this case, we try to choose a point in order to minimize the error function:

$$\begin{aligned} E_{II}(x_0, y_0) &= \int_{x_1}^{x_2} \int_{y_1}^{y_2} \left[\frac{x}{y} - \left(\frac{x_0}{y_0} + \frac{1}{y_0}x - \frac{x_0}{y_0^2}y \right) \right] dx dy \\ &= \frac{x_2^2 - x_1^2}{2}(\ln y_2 - \ln y_1) - \frac{x_0}{y_0}(x_2 - x_1)(y_2 - y_1) \\ &\quad - \frac{1}{y_0} \frac{x_2^2 - x_1^2}{2}(y_2 - y_1) + \frac{x_0}{y_0^2}(x_2 - x_1) \frac{y_2^2 - y_1^2}{2}. \end{aligned} \quad (2)$$

Thus we have

$$\begin{cases} \frac{\partial E_{II}}{\partial x_0} = -\frac{(x_2-x_1)(y_2-y_1)}{y_0} + \frac{1}{y_0^2} \frac{(y_2^2-y_1^2)(x_2-x_1)}{2} = 0; \\ \frac{\partial E_{II}}{\partial y_0} = \frac{x_0}{y_0^2} (x_2-x_1)(y_2-y_1) + \frac{1}{y_0^2} \frac{(x_2^2-x_1^2)(y_2-y_1)}{2} - \frac{x_0}{y_0^3} = 0. \end{cases}$$

It follows from the first equation $\frac{\partial E_{II}}{\partial x_0} = 0$ that we obtain $y_0 = \frac{y_1+y_2}{2} = k_1$. Substitute y_0 into the equation $\frac{\partial E_{II}}{\partial y_0} = 0$, we get $x_0 = \frac{x_1+x_2}{2} = k_2$. Therefore the approximation of $z = \frac{x}{y}$ by the tangent plane at $(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$ is:

$$\begin{aligned} \frac{x}{y} &\approx \frac{x_0}{y_0} + \frac{1}{y_0} x - \frac{x_0}{y_0^2} y = \frac{k_2}{k_1} + \frac{1}{k_1} x - \frac{k_2}{k_1^2} y \\ &= \frac{1}{k_1^2} (k_1 k_2 + k_1 x - k_2 y), \quad \forall (x, y) \in [x_1, x_2] \times [y_1, y_2]. \end{aligned} \quad (3)$$

As in Subsection III-A, we set the approximate linear function $z_{II} = \frac{1}{k_1^2} (k_1 k_2 + k_1 x - k_2 y)$. Comparing the approximation formula z_I in Subsection III-A, we find that these two methods obtain the same approximation formulae for $\frac{x}{y}$ on $[x_1, x_2] \times [y_1, y_2]$. This consistency also explains the rationality of the approximate formula in some sense, and increases our interest in the method. We substitute $x_0 = \frac{x_1+x_2}{2}$, $y_0 = \frac{y_1+y_2}{2}$ into $E_{II}(x_0, y_0)$ in (2) and obtain

$$\begin{aligned} E_{II}(x_0, y_0) &= \frac{x_2^2 - x_1^2}{2} (\ln y_2 - \ln y_1) - \frac{x_2^2 - x_1^2}{y_2 + y_1} (y_2 - y_1) \\ &\quad - \frac{x_2^2 - x_1^2}{y_2 + y_1} (y_2 - y_1) + 2 \frac{x_2^2 - x_1^2}{y_2 + y_1} (y_2 - y_1) \\ &= \frac{x_2^2 - x_1^2}{2} (\ln y_2 - \ln y_1) \neq 0. \end{aligned} \quad (4)$$

So the approximate formula z_{II} is not unbiased.

In the following using a similar computation as above, we can obtain that on $[x_1, x_2] \times [y_1, y_2]$ the approximation of the multiplication $x \cdot y$ by the tangent plane at $(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2} = k_1)$ is:

$$x \cdot y \approx -k_1 k_2 + k_1 x + k_2 y,$$

which is the main result in [1]. Define the implicit surface function:

$$F(x, y, z) = z - xy = 0.$$

Compute the partial derivatives at the point (x_0, y_0) with $z_0 = x_0 y_0$:

$$\frac{\partial F}{\partial z} = 1, \quad \frac{\partial F}{\partial x} = -y_0, \quad \frac{\partial F}{\partial y} = -x_0.$$

Hence, the tangent plane equation at (x_0, y_0) is:

$$-y_0(x - x_0) - x_0(y - y_0) + 1 \cdot (z - z_0) = 0.$$

Rearrange to obtain the tangent plane approximation:

$$\begin{aligned} z &= x_0 y_0 + y_0(x - x_0) + x_0(y - y_0) \\ &= -x_0 y_0 + y_0 x + x_0 y. \end{aligned}$$

We select the optimal tangent plane by minimizing the integral error functional:

$$\begin{aligned} E(x_0, y_0) &= \int_{x_1}^{x_2} \int_{y_1}^{y_2} [xy - (-x_0 y_0 + y_0 x + x_0 y)] dx dy \\ &= \frac{x_2^2 - x_1^2}{2} \cdot \frac{y_2^2 - y_1^2}{2} + x_0 y_0 (x_2 - x_1)(y_2 - y_1) \\ &\quad - y_0 (y_2 - y_1) \cdot \frac{x_2^2 - x_1^2}{2} - x_0 (x_2 - x_1) \cdot \frac{y_2^2 - y_1^2}{2}. \end{aligned}$$

Set the partial derivatives to zero to minimize E :

$$\begin{cases} 0 = \frac{\partial E}{\partial x_0} = y_0(x_2 - x_1)(y_2 - y_1) - (x_2 - x_1) \frac{y_2^2 - y_1^2}{2}; \\ 0 = \frac{\partial E}{\partial y_0} = x_0(x_2 - x_1)(y_2 - y_1) - (y_2 - y_1) \frac{x_2^2 - x_1^2}{2}. \end{cases}$$

We obtain $y_0 = \frac{y_1+y_2}{2} = k_1$ and $x_0 = \frac{x_1+x_2}{2} = k_2$ again. Therefore the approximation of $z = x \cdot y$ by the tangent plane at $(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$ is:

$$\begin{aligned} x \cdot y &\approx -x_0 y_0 + y_0 x + x_0 y \\ &= -k_1 k_2 + k_1 x + k_2 y, \quad \forall (x, y) \in [x_1, x_2] \times [y_1, y_2], \end{aligned}$$

which offers another perspective on the core approximation formula in [1].

C. Architecture for Multi-Level Approximate Divider

In the following of this paper, due to the consistency between First Method and Second Method, we adopt unified notations to denote the approximation of division x/y on $[x_1, x_2] \times [y_1, y_2]$:

$$\frac{x}{y} \approx z_D = \frac{1}{k_1^2} (k_1 k_2 + k_1 x - k_2 y);$$

and the approximation of multiplication $x \cdot y$ on $[x_1, x_2] \times [y_1, y_2]$:

$$x \cdot y \approx z_M = -k_1 k_2 + k_1 x + k_2 y,$$

where $k_1 = \frac{y_1+y_2}{2}$ and $k_2 = \frac{x_1+x_2}{2}$.

According to (4), the finer granularity of partition yields to smaller approximation error. Thus, we propose a multi-level approximation divider architecture which is runtime configurable.

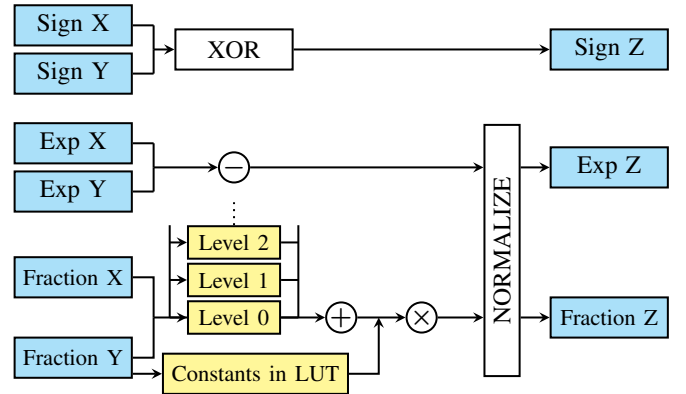


Fig. 1. Architecture of the proposed approximate divider.

As shown in Figure 1, level 0 is denoted as the basic approximation module, which provides an initial estimate, while the deeper levels act as error compensation to gradually improve the overall accuracy. Thus by specifying the desired depths we can realize the run-time configurability.

The initial domain for the mantissas in an FP divider is $[1, 2) \times [1, 2)$. For one level deeper, as shown in Figure 2 we partition each rectangle domain to 2^2 -subregions of rectangle shapes from the middle points as follows:

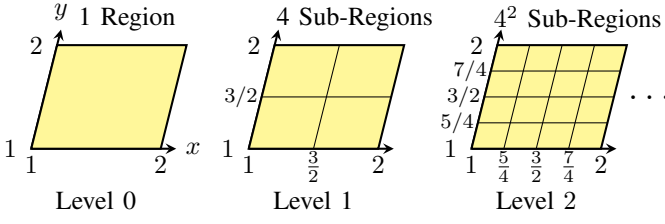


Fig. 2. Multi-level partition of mantissa domain $[1, 2) \times [1, 2)$

level 0: $[1, 2) \times [1, 2)$;
 level 1: $[1, 1.5) \times [1, 1.5)$, $[1, 1.5) \times [1.5, 2)$, $[1.5, 2) \times [1, 1.5)$, $[1.5, 2) \times [1.5, 2)$;
 level n : $[x_{p-1}^n, x_p^n) \times [y_{q-1}^n, y_q^n)$, where $x_p^n = 1 + \frac{p}{2^n}$, $y_q^n = 1 + \frac{q}{2^n}$.

We denote the p th interval for x on the n -level as $[x_{p-1}^n, x_p^n)$ for $p = 1, \dots, 2^n$ and similarly the q th interval for y on the n -level as $[y_{q-1}^n, y_q^n)$ for $q = 1, \dots, 2^n$. Thus the interval middle points are

$$\widehat{x_p^n} = x_{p-1}^n + \frac{1}{2^{n+1}} = 1 + \frac{p-1}{2^n} + \frac{1}{2^{n+1}} = 1 + \frac{2p-1}{2^{n+1}} = k_2$$

$$\widehat{y_q^n} = y_{q-1}^n + \frac{1}{2^{n+1}} = 1 + \frac{q-1}{2^n} + \frac{1}{2^{n+1}} = 1 + \frac{2q-1}{2^{n+1}} = k_1.$$

Therefore by equation (3), for any level n we can derive the following:

$$z_D^n = \frac{1}{(\widehat{y_{q(n)}^n})^2} (\widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} + \widehat{y_{q(n)}^n} \cdot x - \widehat{x_{p(n)}^n} \cdot y), \quad (5)$$

where $p(n), q(n)$ are the indexes to specify the subregion which (x, y) belongs to in the n th level. In the following, we will implement $\frac{1}{(\widehat{y_{q(n)}^n})^2}$ and $w_n = \widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} + \widehat{y_{q(n)}^n} \cdot x - \widehat{x_{p(n)}^n} \cdot y$ separately.

(i) **First for w_n .** The following method is similar to that for approximate multiplier in [1]. For convenience of readers, we list main calculations of this method. We have

$$\begin{aligned} w_n &= \widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} + \widehat{y_{q(n)}^n} \cdot x - \widehat{x_{p(n)}^n} \cdot y \\ &= (w_n - w_{n-1}) + (w_{n-1} - w_{n-2}) + \dots + (w_1 - w_0) + w_0 \\ &= \sum_{i=1}^n \Delta w_i + \widehat{x_{p(0)}^0} \cdot \widehat{y_{q(0)}^0} + \widehat{y_{q(0)}^0} \cdot x - \widehat{x_{p(0)}^0} \cdot y \\ &= \sum_{i=1}^n \Delta w_i + 2.25 + 1.5x - 1.5y. \end{aligned} \quad (6)$$

It is easy to calculate that

$$\widehat{x_{p(0)}^0} = \frac{3}{2};$$

and

$$\widehat{x_{p(1)}^1} = \frac{3}{2} + \frac{1}{2^2} \text{ if } x > \widehat{x_{p(0)}^0};$$

$$\widehat{x_{p(1)}^1} = \frac{3}{2} - \frac{1}{2^2} \text{ if } x < \widehat{x_{p(0)}^0};$$

i.e.

$$\widehat{x_{p(1)}^1} = \widehat{x_{p(0)}^0} + \frac{x[1]?(1) : (-1)}{2^2}.$$

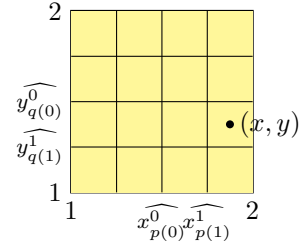


Fig. 3. Illustration of the point (x, y) in the partitioned domain.

Thus in general, we have

$$\widehat{x_{p(n)}^n} = \widehat{x_{p(n-1)}^{n-1}} + \frac{x[n]?(1) : (-1)}{2^{n+1}}$$

and similarly

$$\widehat{y_{q(n)}^n} = \widehat{y_{q(n-1)}^{n-1}} + \frac{y[n]?(1) : (-1)}{2^{n+1}},$$

where $x[n], y[n]$ are the n th bit after decimal point in binary expansion of x, y separately. Thus

$$\begin{aligned} \Delta w_n &= [\widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} + \widehat{y_{q(n)}^n} \cdot x - \widehat{x_{p(n)}^n} \cdot y] \\ &\quad - [\widehat{x_{p(n-1)}^{n-1}} \cdot \widehat{y_{q(n-1)}^{n-1}} + \widehat{y_{q(n-1)}^{n-1}} \cdot x - \widehat{x_{p(n-1)}^{n-1}} \cdot y] \\ &= [\widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} - \widehat{x_{p(n-1)}^{n-1}} \cdot \widehat{y_{q(n-1)}^{n-1}}] \\ &\quad + (\widehat{y_{q(n)}^n} - \widehat{y_{q(n-1)}^{n-1}})x - (\widehat{x_{p(n)}^n} - \widehat{x_{p(n-1)}^{n-1}})y \\ &= [(\widehat{x_{p(n-1)}^{n-1}} + \frac{x[n]?(1) : (-1)}{2^{n+1}}) \cdot (\widehat{y_{q(n-1)}^{n-1}} + \frac{y[n]?(1) : (-1)}{2^{n+1}}) \\ &\quad - \widehat{x_{p(n-1)}^{n-1}} \cdot \widehat{y_{q(n-1)}^{n-1}}] \\ &\quad + \frac{y[n]?(1) : (-1)}{2^{n+1}}x - \frac{x[n]?(1) : (-1)}{2^{n+1}}y \\ &= (x + \widehat{x_{p(n-1)}^{n-1}}) \cdot \frac{y[n]?(1) : (-1)}{2^{n+1}} \\ &\quad + (\widehat{y_{q(n-1)}^{n-1}} - y) \cdot \frac{x[n]?(1) : (-1)}{2^{n+1}} \\ &\quad + \frac{x[n]?(1) : (-1)}{2^{n+1}} \cdot \frac{y[n]?(1) : (-1)}{2^{n+1}} \\ &= (x + \widehat{x_{p(n-1)}^{n-1}}) \cdot (y[n]?(1) : (-1)) \gg n+1 \\ &\quad + (\widehat{y_{q(n-1)}^{n-1}} - y) \cdot (x[n]?(1) : (-1)) \gg n+1 \\ &\quad + (x[n] \wedge y[n])?(-1) : (1) \gg 2n+2. \end{aligned} \quad (7)$$

Note that $\widehat{x_{p(n-1)}^{n-1}}, \widehat{y_{q(n-1)}^{n-1}}$ are the middle points of the intervals that contain x, y separately. $(x + \widehat{x_{p(n-1)}^{n-1}}) \cdot (y[n]?(1) : (-1)) \gg n+1$ can be implemented in circuit with 1 addition operation, at most 1 inversion operation and 1 $(n+1)$ -bits right shift operation; $(\widehat{y_{q(n-1)}^{n-1}} - y) \cdot (x[n]?(1) : (-1)) \gg n+1$ can be implemented in circuit with logic operations by turning the first n bits in y to 0 and bit flipping (if necessary) for the rest of bits in y , at most 1 inversion operation and 1 $(n+1)$ -bits right shift operation; and $(x[n] \wedge y[n])?(-1) : (1) \gg 2n+2$ can be implemented with 1 XOR operation, at most 1 inversion operation and 1 $(2n+2)$ -bits right shift operation.

(ii) **Secondly** for $\frac{1}{(\widehat{y_{q(n)}^n})^2} \cdot \frac{1}{(\widehat{y_{q(n)}^n})^2}$ is constant that can be pre-calculated and pre-loaded lookup tables for each level n and $q(n) = 1, 2, \dots, 2^n$. Since

$$\begin{aligned}\widehat{y_{q(n)}^n} &= \widehat{y_{q(n-1)}^{n-1}} + \frac{y[n]?1: (-1)}{2^{n+1}} \\ &= \widehat{y_{q(n-2)}^{n-2}} + \frac{y[n-1]?1: (-1)}{2^n} + \frac{y[n]?1: (-1)}{2^{n+1}}, \\ &= \dots = \widehat{y_{q(0)}^0} + \sum_{k=1}^n \frac{y[k]?1: (-1)}{2^{k+1}}\end{aligned}$$

we list the constants $\frac{1}{(\widehat{y_{q(n)}^n})^2}$ for level 0, 1, 2, 3 as follows:

$$\text{level 0 : } \frac{1}{(\widehat{y_{q(0)}^0})^2} = \frac{1}{(3/2)^2}$$

$$\text{level 1 : } \frac{1}{(\widehat{y_{q(1)}^1})^2} = \begin{cases} \frac{1}{(7/4)^2} & \text{if } y[1] = 1; \\ \frac{1}{(5/4)^2} & \text{if } y[1] = 0; \end{cases}$$

$$\text{level 2 : } \frac{1}{(\widehat{y_{q(2)}^2})^2} = \begin{cases} \frac{1}{(15/8)^2} & y[1] = 1, y[2] = 1; \\ \frac{1}{(13/8)^2} & y[1] = 1, y[2] = 0; \\ \frac{1}{(11/8)^2} & y[1] = 0, y[2] = 1; \\ \frac{1}{(9/8)^2} & y[1] = 0, y[2] = 0; \end{cases}$$

$$\text{level 3 : } \frac{1}{(\widehat{y_{q(3)}^3})^2} = \begin{cases} \frac{1}{(31/16)^2} & y[1] = 1, y[2] = 1, y[3] = 1; \\ \frac{1}{(29/16)^2} & y[1] = 1, y[2] = 1, y[3] = 0; \\ \frac{1}{(27/16)^2} & y[1] = 1, y[2] = 0, y[3] = 1; \\ \frac{1}{(25/16)^2} & y[1] = 1, y[2] = 0, y[3] = 0; \\ \frac{1}{(23/16)^2} & y[1] = 0, y[2] = 1, y[3] = 1; \\ \frac{1}{(21/16)^2} & y[1] = 0, y[2] = 1, y[3] = 0; \\ \frac{1}{(19/16)^2} & y[1] = 0, y[2] = 0, y[3] = 1; \\ \frac{1}{(17/16)^2} & y[1] = 0, y[2] = 0, y[3] = 0; \end{cases}$$

$y[0] \ [1] \ [2] \ [3] \ [4] \ [5] \ \dots$

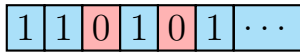


Fig. 4. An example y in $[1, 2)$.

For example, mantissa $y = \overline{1.10101\dots}$ (see Figure 4), we have

$$\begin{aligned}\frac{1}{(\widehat{y_{q(0)}^0})^2} &= \frac{1}{(3/2)^2}, \\ \frac{1}{(\widehat{y_{q(1)}^1})^2} &= \frac{1}{(\frac{3}{2} + \frac{1}{4})^2} = \frac{1}{(7/4)^2}, \\ \frac{1}{(\widehat{y_{q(2)}^2})^2} &= \frac{1}{(\frac{3}{2} + \frac{1}{4} - \frac{1}{8})^2} = \frac{1}{(13/8)^2}, \\ \frac{1}{(\widehat{y_{q(3)}^3})^2} &= \frac{1}{(\frac{3}{2} + \frac{1}{4} - \frac{1}{8} + \frac{1}{16})^2} = \frac{1}{(27/16)^2}, \\ \frac{1}{(\widehat{y_{q(4)}^4})^2} &= \frac{1}{(\frac{3}{2} + \frac{1}{4} - \frac{1}{8} + \frac{1}{16} - \frac{1}{32})^2} = \frac{1}{(53/32)^2}.\end{aligned}$$

D. Architecture for Multi-Level Approximate Divider-Multiplier

It follows from [1] that for any level n we have the following:

$$\begin{aligned}z_M^n &= -\widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} + \widehat{y_{q(n)}^n} \cdot x + \widehat{x_{p(n)}^n} \cdot y \\ &= \sum_{i=1}^n \Delta z_i - \widehat{x_{p(0)}^0} \cdot \widehat{y_{q(0)}^0} + \widehat{y_{q(0)}^0} \cdot x + \widehat{x_{p(0)}^0} \cdot y \\ &= \sum_{i=1}^n \Delta z_i - 2.25 + 1.5x + 1.5y,\end{aligned}\tag{8}$$

where

$$\begin{aligned}\Delta z_n &= [-\widehat{x_{p(n)}^n} \cdot \widehat{y_{q(n)}^n} + \widehat{y_{q(n)}^n} \cdot x + \widehat{x_{p(n)}^n} \cdot y] \\ &\quad - [-\widehat{x_{p(n-1)}^{n-1}} \cdot \widehat{y_{q(n-1)}^{n-1}} + \widehat{y_{q(n-1)}^{n-1}} \cdot x + \widehat{x_{p(n-1)}^{n-1}} \cdot y] \\ &= (x - \widehat{x_{p(n-1)}^{n-1}}) \cdot (y[n]?(1) : (-1)) \gg n + 1 \\ &\quad + (y - \widehat{y_{q(n-1)}^{n-1}}) \cdot (x[n]?(1) : (-1)) \gg n + 1 \\ &\quad + (x[n]^\wedge y[n]?(1) : (-1)) \gg 2n + 2.\end{aligned}\tag{9}$$

As mentioned in [1], $(x - \widehat{x_{p(n-1)}^{n-1}}) \cdot (y[n]?(1) : (-1)) \gg n + 1$ and $(y - \widehat{y_{q(n-1)}^{n-1}}) \cdot (x[n]?(1) : (-1)) \gg n + 1$ can be implemented in circuit with logic operations by turning the first n bits in x, y to 0 and bit flipping (if necessary) for the rest of bits in x, y , at most 1 inversion operation and 1 $(n+1)$ -bits right shift operation; and $(x[n]^\wedge y[n]?(1) : (-1)) \gg 2n + 2$ can be implemented with 1 XOR operation, at most 1 inversion operation and 1 $(2n+2)$ -bits right shift operation.

Comparing Eqs. (7) with (9), we arrive at the following relations:

$$\Delta w_n = -\Delta z_n + 2x \cdot (y[n]?(1) : (-1)) \gg n + 1 \tag{10}$$

$$w_0 = -z_M^0 + 3x, \tag{11}$$

where $w_0 = 2.25 + 1.5x - 1.5y$ and $z_M^0 = -2.25 + 1.5x + 1.5y$; and

$$\begin{aligned}&(x \pm \widehat{x_{p(n-1)}^{n-1}}) \cdot (y[n]?(1) : (-1)) \gg n + 1 \\ &\mp [(y - \widehat{y_{q(n-1)}^{n-1}}) \cdot (x[n]?(1) : (-1)) \gg n + 1 \\ &+ (x[n]^\wedge y[n]?(1) : (-1)) \gg 2n + 2] = \begin{cases} \Delta w_n; \\ \Delta z_n; \end{cases}\end{aligned}\tag{12}$$

$$1.5x \mp (1.5y - 2.25) = \begin{cases} w_0; \\ z_M^0. \end{cases}\tag{13}$$

Motivated by the similarity between z_M^n and the main part w_n of z_D^n , we propose a unified hardware architecture supporting both approximate floating-point division and multiplication as shown in Figure 5.

IV. ERROR ANALYSIS

In this section, we further discuss the absolute error for different levels of approximations in the proposed divider. We recall the Taylor's Theorem for two variables: for $0 < \theta < 1$,

$$\begin{aligned}f(x, y) &= f(x_0, y_0) + [(x - x_0) \frac{\partial}{\partial x} + (y - y_0) \frac{\partial}{\partial y}] f(x_0, y_0) \\ &\quad + [(x - x_0) \frac{\partial}{\partial x} + (y - y_0) \frac{\partial}{\partial y}]^2 f(x_0 + \theta(x - x_0), y_0 + \theta(y - y_0)).\end{aligned}$$

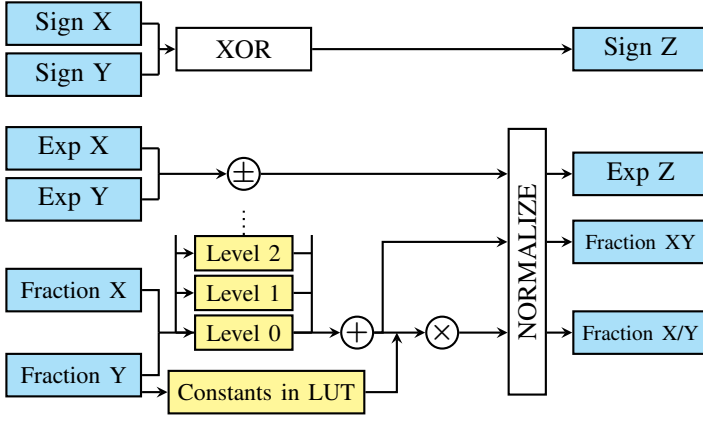


Fig. 5. Architecture of the proposed approximate divider-multiplier.

For $f(x, y) = \frac{x}{y}$ on $[x_1, x_2] \times [y_1, y_2]$ and $x_0 = \frac{x_1+x_2}{2} = k_2$, $y_0 = \frac{y_1+y_2}{2} = k_1$ as the symbols in Section III, we have

$$\begin{aligned} \frac{x}{y} &= \frac{x_0}{y_0} + (x - x_0) \cdot \frac{1}{y_0} + (y - y_0) \cdot \left(-\frac{x_0}{y_0^2}\right) \\ &+ \frac{1}{2} \left[(x - x_0)^2 \cdot 0 + 2(x - x_0)(y - y_0) \left(-\frac{1}{(y_0 + \theta(y - y_0))^2}\right) \right. \\ &\left. + (y - y_0)^2 \cdot \frac{2(x_0 + \theta(x - x_0))}{(y_0 + \theta(y - y_0))^3} \right] \\ &= \frac{1}{k_1^2} (k_1 k_2 + k_1 x - k_2 y) \\ &+ \frac{(y - y_0)[(y - y_0)(x_0 + \theta(x - x_0)) - (x - x_0)(y_0 + \theta(y - y_0))]}{(y_0 + \theta(y - y_0))^3}. \end{aligned} \quad (14)$$

It follows from equation (3) and (14) that

$$\frac{x}{y} = z_D + err, \text{ for } (x, y) \in [x_1, x_2] \times [y_1, y_2],$$

where err is

$$\frac{(y - y_0)[(y - y_0)(x_0 + \theta(x - x_0)) - (x - x_0)(y_0 + \theta(y - y_0))]}{(y_0 + \theta(y - y_0))^3}. \quad (15)$$

Thus for the n th level and $(x, y) \in [x_{p(n)-1}^n, x_{p(n)}^n] \times [y_{q(n)-1}^n, y_{q(n)}^n]$, where $x_{p(n)}^n = 1 + \frac{p(n)}{2^n}$ and $y_{q(n)}^n = 1 + \frac{q(n)}{2^n}$, it follows from the equation (15) that

$$\begin{aligned} |err| &\leq \frac{|y - y_0|(2|y - y_0| + 2|x - x_0|)}{1^3} \\ &\leq \frac{1}{2^{n+1}} \cdot \frac{4}{2^{n+1}} = \frac{1}{4^n}. \end{aligned} \quad (16)$$

V. EXPERIMENTAL RESULTS

A. Circuit Implementation for Multi-Level

For $x, y \in [1, 2)$, it follows from Equations (5), (6), (8), (12) and (13) that

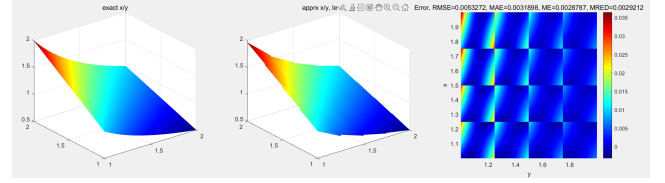


Figure 6 Level 2 Errors

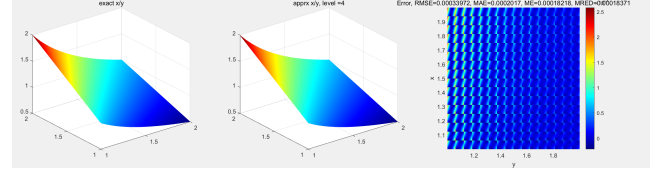


Figure 7 Level 4 Errors

$$\text{level 0 : } \frac{x}{y} \approx z_D^0 = \frac{1}{(\widehat{y_{q(0)}})^2} w_0;$$

$$xy \approx z_M^0;$$

$$\text{level 1 : } \frac{x}{y} \approx z_D^1 = \frac{1}{(\widehat{y_{q(1)}})^2} (\Delta w_1 + w_0);$$

$$xy \approx z_M^1 = \Delta z_1 + z_M^0;$$

$$\text{level 2 : } \frac{x}{y} \approx z_D^2 = \frac{1}{(\widehat{y_{q(2)}})^2} (\Delta w_2 + \Delta w_1 + w_0);$$

$$xy \approx z_M^2 = \Delta z_2 + \Delta z_1 + z_M^0;$$

.....

where

$$w_0 \text{ or } z_M^0 = 1.5x \mp (1.5y - 2.25);$$

$$\begin{aligned} \Delta w_1 \text{ or } \Delta z_1 &= (x \pm \widehat{x_{p(0)}}^0) \cdot (y[1]?(1) : (-1)) >> 2 \\ &\mp [(y - \widehat{y_{q(0)}}^0) \cdot (x[1]?(1) : (-1)) >> 2 \\ &+ (x[1]^{\wedge} y[1]?(1) : (-1)) >> 4]; \end{aligned}$$

$$\begin{aligned} \Delta w_2 \text{ or } \Delta z_2 &= (x \pm \widehat{x_{p(1)}}^1) \cdot (y[2]?(1) : (-1)) >> 3 \\ &\mp [(y - \widehat{y_{q(1)}}^1) \cdot (x[2]?(1) : (-1)) >> 3 \\ &+ (x[2]^{\wedge} y[2]?(1) : (-1)) >> 6]; \end{aligned}$$

.....

This section further presents a concrete multi-level architecture designed to implement these algorithms (divide or multiple) for $x, y \in [1, 2)$, allowing for dynamic configuration at run-time, which is illustrated in Figure 8. The circuit implementation of $\Delta w_n, \Delta z_n$ is detailed in subsection III-C and III-D. The proposed approximate divider organically combines the multi-level architecture with the LUT-based design.

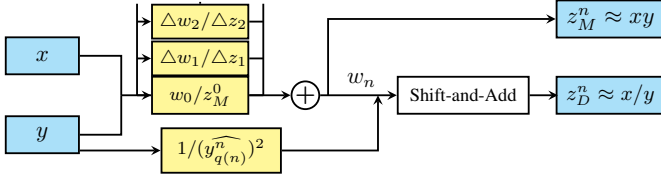


Fig. 8. Concrete architecture of the approximate divider-multiplier.

VI. APPLICATIONS

We evaluated the proposed divider across different approximate levels, focusing on accuracy, resource utilization, and performance. This section is dedicated to examining the disparities between the outcomes of the proposed approximate divider and the exact divider in practical applications. We conduct several benchmark tests, include image processing approaches such as change detection, background removal, K-Mean image color quantization, and Softmax layers in neural networks.

VII. CONCLUSIONS

In this work, we proposed a design for multi-level approximate FP divider with runtime configurability. The architecture can reach the optimal approximation and unbiased error distribution easily without increasing much area and delay overhead.

REFERENCES

- [1] C.T. Chen, et al., "Optimally Approximated and Unbiased Floating Point Multiplier with Runtime Configurability," International Conference on Computer-Aided Design (ICCAD), pp.1-9, 2020.
- [2] J. Han and M. Orshansky, "Approximate Computing: An Emerging Paradigm for Energy-Efficient Design," European Test Symposium(ETS), pp.1-6, 2013.
- [3] V. Gupta, et al., "IMPACT: IMPrecise Adders for Low-Power Approximate Computing," International Symposium on Low Power Electronics and Design (ISLPED), pp. 409-414, 2011.
- [4] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol.32, no. 1, pp.124-137, 2013.
- [5] S. Hashemi, R. I. Bahar, and S. Reda, "Drum: A dynamic range unbiased multiplier for approximate applications," International Conference on Computer-Aided Design(ICCAD), pp.418-425, 2015.
- [6] H. Mahdiani, A. Ahmadi, S. Fakhraie, and C. Lucas, "Bio-inspired imprecise computational blocks for efficient VLSI implementation of soft-computing applications," IEEE Transactions on Circuits and Systems I, vol.57, no.4, pp.850-862, 2010.
- [7] H. Saadat et al., "Approximate Integer and Floating-Point Dividers with Near-Zero Error Bias," Design Automation Conference (DAC), pp.1-6, 2019.
- [8] L. Chen et al., "Design of Approximate Unsigned Integer Non-restoring Divider for Inexact Computing," Great Lakes Symposium on VLSI (GLSVLSI), pp.51-56, 2015.
- [9] S. Hashemi et al., "A low-power dynamic divider for approximate applications," Design Automation Conference (DAC), pp.1-6, 2016.
- [10] H. Jiang et al., "Low-Power Unsigned Divider and Square Root Circuit Designs Using Adaptive Approximation," IEEE Trans.on Computers, vol.68, no.11, pp.1635-1646, 2019.
- [11] M.Imani et al., "CADE: Configurable Approximate Divider for Energy Efficiency," Design, Automation, Test in Europe Conference and Exhibition (DATE), pp.586-589, 2019.
- [12] O.G. Ratnaparkhi et al., "LEAD: Logarithmic Exponent Approximate Divider For Image Quantization Application," Great Lakes Symposium on VLSI (GLSVLSI), pp.437-442, 2022.
- [13] J. Melchert et al., "SAADI-EC: A Quality-Configurable Approximate Divider for Energy Efficiency," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol.27, no.11, pp.2680-2682, 2019.
- [14] J. Oelund et al., "ILAFD: Accuracy-Configurable Floating-Point Divider Using an Approximate Reciprocal and an Iterative Logarithmic Multiplier," Great Lakes Symposium on VLSI (GLSVLSI), pp.639-644, 2023.
- [15] K.J.N.S. Bhargav et al., "A newton raphson method based approximate divider design for color quantization application," in ISOCC'21, pp.115-116.
- [16] C. Wen et al., "PACE: A Piece-Wise Approximate and Configurable Floating-Point Divider for Energy-Efficient Computing," in DATE'2024.
- [17] H. Liu et al., "QIAD: A quadratic interpolation approximate divider," IEICE Electronics Express, vol.20, no.11, pp.1-6, 2023.
- [18] L. Wu et al., "A curve fitting approach for non-iterative divider design with accuracy and performance trade-off," International New Circuits and Systems Conference (NEWCAS), pp.1-4, 2015.
- [19] C. Wu et al., "Area-Delay-Energy-Efficient Approximate Dividers Based on Piecewise Linear Fitting of Surface," IEEE Transactions on Circuits and Systems, vol. 71, no. 11, pp. 5017-5029, 2024.
- [20] IEEE Standard for Floating-Point Arithmetic, IEEE Standard 754-2008, pp. 1-70, 2008.
- [21] https://en.wikipedia.org/wiki/Division_algorithm